CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

Team Members: Jamison Bice; Christopher Funk

Project Teams Group #: D_01

2 RISC-V Single-cycle Processor

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.

The schematic is included in Appendix A.

3 Component Implementation

3.1 Control Logic

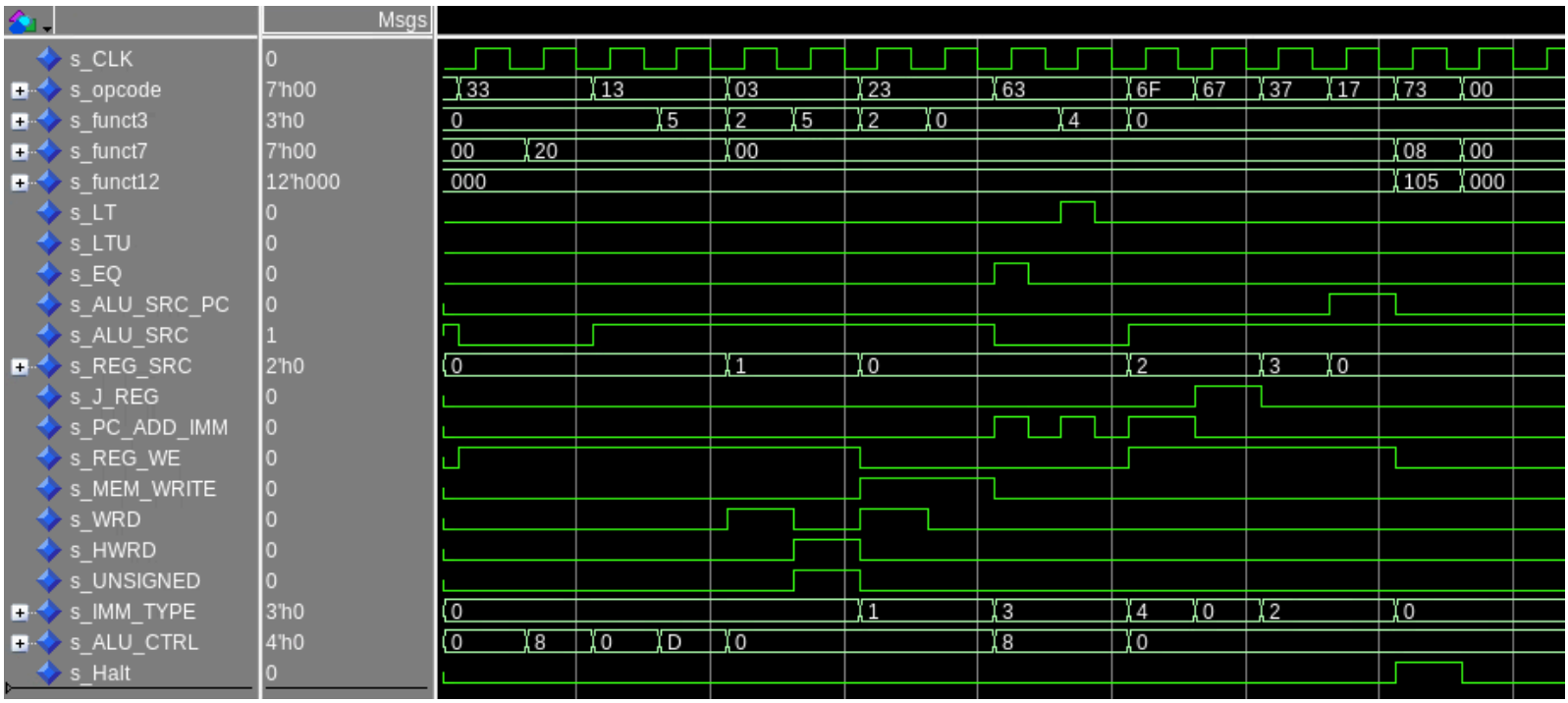
[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an *N*M* table where each row corresponds to the output of the control logic module for a given instruction.

Link to the spreadsheet:

https://docs.google.com/spreadsheets/d/1asLtcapO9Zwo_TIthWWjVvxY27MOP9UHLt8TDI8ZGN4/edit?usp=sharing

This spreadsheet is also part of the submission.

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).



Cycle 1 is **add,** which matches: opcode=0x33=0110011, funct3=0,funct7=0. Output: s_REG_WE =1 everything else stays the same at 0.

Cycle 2 is **sub**, which matches: opcode = 0x33 = 0110011, funct3 = 0 (000), funct7 = 0x20 (0100000). Output: s_REG_WE = 1 and s_ALU_CTRL = 8 (1000), while everything else stays at 0.

Cycle 3 is **addi**, which matches: opcode = 0x13 = 0010011, funct3 = 0 (000). Output: s_ALU_SRC = 1 and s_REG_WE = 1, while everything else stays at 0

Cycle 4 is **srai**, which matches: opcode = 0x13 = 0010011, funct3 = 5 (101), funct7 = 0x20 (0100000). Output: s_ALU_SRC = 1, s_REG_WE = 1, and s_ALU_CTRL = D (1101), while everything else stays at 0.

Cycle 5 is **lw**, which matches: opcode = 0x03 = 0000011, funct3 = 2 (010). Output: s_ALU_SRC = 1, s_REG_SRC = 1 (01), s_REG_WE = 1, and s_WRD = 1, while everything else stays at 0.

Cycle 6 is **lhu**, which matches: opcode = 0x03 = 0000011, funct3 = 5 (101). Output: s_ALU_SRC = 1, s_REG_SRC = 1 (01), s_REG_WE = 1, s_HWRD = 1, and s_UNSIGNED = 1, while everything else stays at 0.

Cycle 7 is **sw**, which matches: opcode = 0x23 = 0100011, funct3 = 2 (010). Output: s_ALU_SRC = 1, s_MEM_WRITE = 1, s_WRD = 1, and s_IMM_TYPE = 1 (001), while everything else stays at 0.

Cycle 8 is **sb**, which matches: opcode = 0x23 = 0100011, funct3 = 0 (000). Output: s_ALU_SRC = 1, s_MEM_WRITE = 1, and s_IMM_TYPE = 1 (001), while everything else stays at 0.

Cycle 9 is **beq**, which matches: opcode = 0x63 = 1100011, funct3 = 0 (000). Output: s_IMM_TYPE = 3 (011) and s_ALU_CTRL = 8 (1000). When input s_EQ = 1 in the first half of the cycle, s_PC_ADD_IMM = 1 (branch taken); when s_EQ = 0 in the second half, s_PC_ADD_IMM returns to 0, while everything else stays at 0.

Cycle 10 is **blt**, which matches: opcode = 0x63 = 1100011, funct3 = 4 (100). Output: s_IMM_TYPE = 3 (011) and s_ALU_CTRL = 8 (1000). When input s_LT = 1 in the first half of the cycle, s_PC_ADD_IMM = 1 (branch taken); when s_LT = 0 in the second half, s_PC_ADD_IMM returns to 0, while everything else stays at 0.

Cycle 11 is **jal**, which matches: opcode = 0x6F = 1101111. Output: s_ALU_SRC = 1, s_REG_SRC = 2 (10), s_PC_ADD_IMM = 1, s_REG_WE = 1, and s_IMM_TYPE = 4 (100), while everything else stays at 0.

Cycle 12 is **jalr**, which matches: opcode = 0x67 = 1100111, funct3 = 0 (000). Output: s_ALU_SRC = 1, s_REG_SRC = 2 (10), s_J_REG = 1, and s_REG_WE = 1, while everything else stays at 0.

Cycle 13 is **lui**, which matches: opcode = 0x37 = 0110111. Output: s_ALU_SRC = 1, s_REG_SRC = 3 (11), s_REG_WE = 1, and s_IMM_TYPE = 2 (010), while everything else stays at 0.

Cycle 14 is **auipc**, which matches: opcode = 0x17 = 0010111. Output: s_ALU_SRC_PC = 1, s_ALU_SRC = 1, s_REG_WE = 1, and s_IMM_TYPE = 2 (010), while everything else stays at 0.

Cycle 15 is **wfi** (Halt), which matches: opcode = 0x73 = 1110011, funct3 = 0 (000), funct12 = 0x105 (000100000101). Output: s_Halt = 1 while everything else stays at 0.

Cycle 16 is Default / NOP, so all baseline 0.

3.2 Fetch Logic

[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

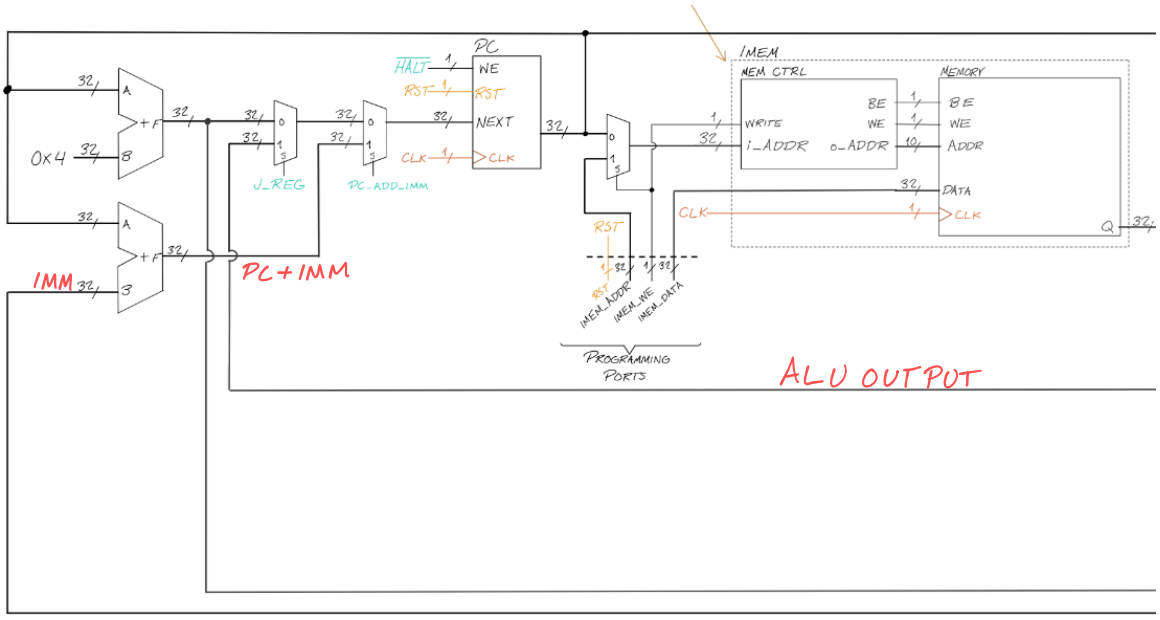
Sequential Execution ($PC_{next} = PC + 4$)

PC-Relative Unconditional Jump ($PC_{next} = PC + imm_{UJ}$)

PC-Relative Conditional Branch ($PC_{next} = PC + imm_{SB}$ if condition met, else $PC + 4$)

Register-Relative Indirect Jump ($PC_{next} = rs1 + imm_{I}$)

These control flow possibilities can also be seen from the Fetch portion of our processor diagram:



[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?

Two dedicated 32-bit adders: ADD_4_ADDER and ADD_IMM_ADDER

Two cascaded 2-to-1 32-bit multiplexers PC_J_REG_MUX and PC_ADD_IMM_MUX

Write-Back & ALU Input Multiplexer Extensions: REG_SRC_MUX, ALU_SRC_PC_MUX

Additional Control Signals Needed:

o_PC_ADD_IMM

o_J_REG

o_ALU_SRC_PC

o_Halt

o_REG_SRC

o_IMM_TYPE

i_EQ

i_LT

i_LTU

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your write-up.

In our processor design, the fetch logic is not a functional unit; instead, it consists of several muxes, adders, and a 32-bit PC register. The logic signals that test the control flow operation were tested with the control unit and were already tested in the instruction decoder; additionally, the control flow instructions such as BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL, and JALR were verified with the control logic module (see waveform above) and test programs in the provided test suite.

3.3 Complete RISC-V ALU

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?

The srl instruction does not fill in the leftmost bits with sign-extended values when shifting right; however, the sra does. A sla instruction wouldn't make sense, since shifting to the left means that bits are added to the right side of the binary number and the rightmost bits do not indicate sign.

[Part 3.3.1.(b)] In your write-up, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

Since we chose to implement the barrel shifter as a logarithmic shifter to shift 32-bits, this logarithmic shifter uses 5 fixed-width shifters connected in series. The VHDL we created for this fixed-width shifter is shown below:

```
       -- sign DATA_WIDTH
        with i_SIGNED select
            s_SIGN_BITS <=  (others => i_DATA(DATA_WIDTH-1)) when
'1', -- duplicate last bit when signed
                           (others => '0') when others; -- otherwise
use zero

        -- shifted signal
        s_SHIFTED_DATA <= s_SIGN_BITS & i_DATA(DATA_WIDTH - 1 downto
SHIFT);
```

```
-- create a N-bit 2-to-1 bus mux
BUS_MUX_2TO1: BUS_MUX_NTO1
 generic map(
    DATA_WIDTH => DATA_WIDTH,
    N => 2,
    LOG2_N => 1
)
 port map(
    i_S => (0 => i_SHIFT),
    i_DATA => (0 => i_DATA, 1 => s_SHIFTED_DATA),
    o_DATA => o_DATA
);
```

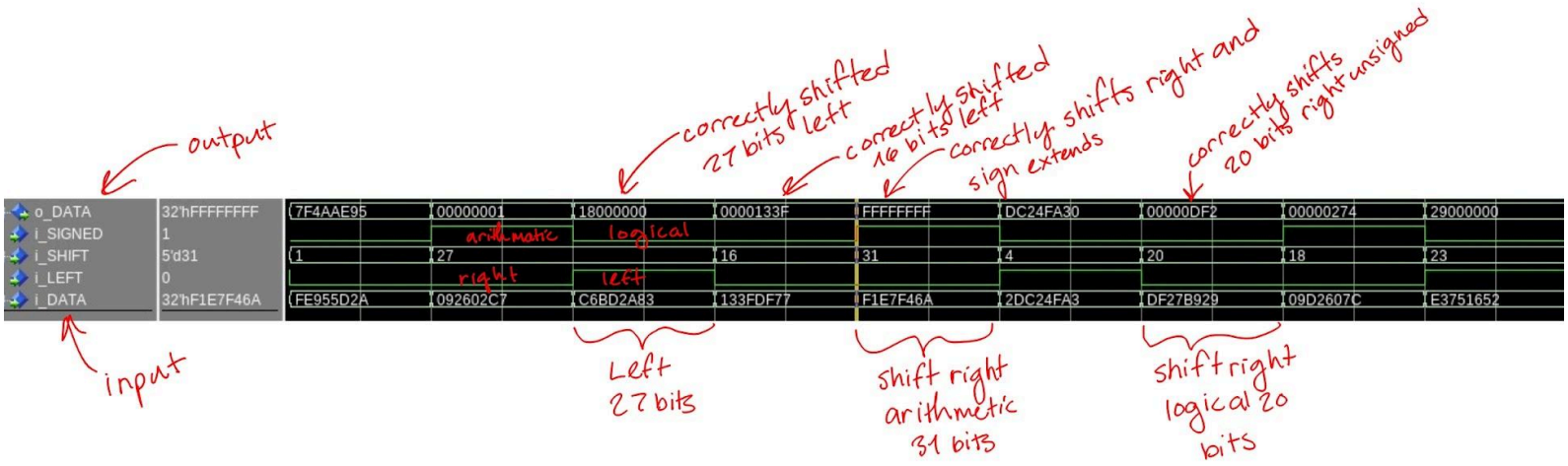
As shown in this code, the input i_SIGNED acts as an input to a mux that determines whether the upper bits in the shifter will be zero or the same sign as the most significant bit in the input vector.

[Part 3.3.1.(c)] In your write-up, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

To enable the Barrel shifter to perform both right and left shifts, I added a “flip” component that reverses the input bit order when the select signal is high. Placing one of these bit-flippers on the input and the output and connecting the left-shift signal to both the bit flippers' select lines allows the logarithmic right shifter to also be used to perform the left shift operation.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your write-up.

I adapted the ALU testbench to test just the shifter. The annotated output for the shifter testbench is shown below.



[Part 3.3.2.(a)] In your write-up, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

Adder/Subtracter

The design of the adder/subtracter could be largely copied and pasted from lab2; however, to accommodate the “less than flag,” an overflow output was added to the adder. This overflow bit is computed by taking the XOR of the adder’s last two carry bits.

Flow Control Flags and Slt and Sltu

Rather than creating a separate control unit for the branching instructions, we chose to integrate the “equal flag” (also called “zero flag”), the “less than” flag, and the “less than unsigned” flag in the ALU. I referenced Stoychev’s ALU slides on the CPRE281 website when correctly calculating these flags using the negative flag, the overflow flag, and the output of the add/subtract unit.

Left/Right Shifter

To build the shifter, my first approach was to create a 32-to-1 32-bit bus mux with all 32 possible shifts as inputs; the shift amount would then select between them. This seemed a bit too large to me, so I googled different barrel shifter architectures, and the AI overview mentioned the logarithmic shifter. For a 32 bit implementation, 5 fixed-bit shifters are connected in series; each one of these five shifters shifts its input by a power of 2. For example, the shifter connected to the 0th bit of the “shift amount” input only shifts one bit to the right, whereas the shifter connected to the 3rd bit shifts its input by 16. Since these shifters are connected in series, if the 0th and 3rd bit of the shift amount signal are asserted, then the input signal will be shifted by 17 bits. This

logarithmic shifter design should take up significantly less area since it only has one 32-bit input rather than having 32 32-bit inputs.

ALU_CTRL Signal

When deciding what values of the 4-bit ALU_CTRL signal should select which ALU function, I determined that, to simplify the control unit's design, the lower three bits of the ALU_CTRL signal should match the func3 signal, and the upper bit should be taken from the func7's bit 5 (assuming bit 6 to be the most significant bit), since this func7 bit is used to indicate whether an arithmetic function is subtracted or, in the case of sra, sign-extended.

The mapping for the ALU_CTRL signal, how the ALU's mathematical operations map to its various input combinations, is shown in the table below (Verilog vector notation is used for consistency with the green card).

ALU_CTRL[3]	Operation
1	SHIFT_RIGHT_SIGNED or SUBTRACT
1	UNSIGNED or ADD
ALU_CTRL[2:0]	Operation
001	SLL
010	SLT
011	SLTU
100	SLTU
101	XOR
110	OR
111	AND

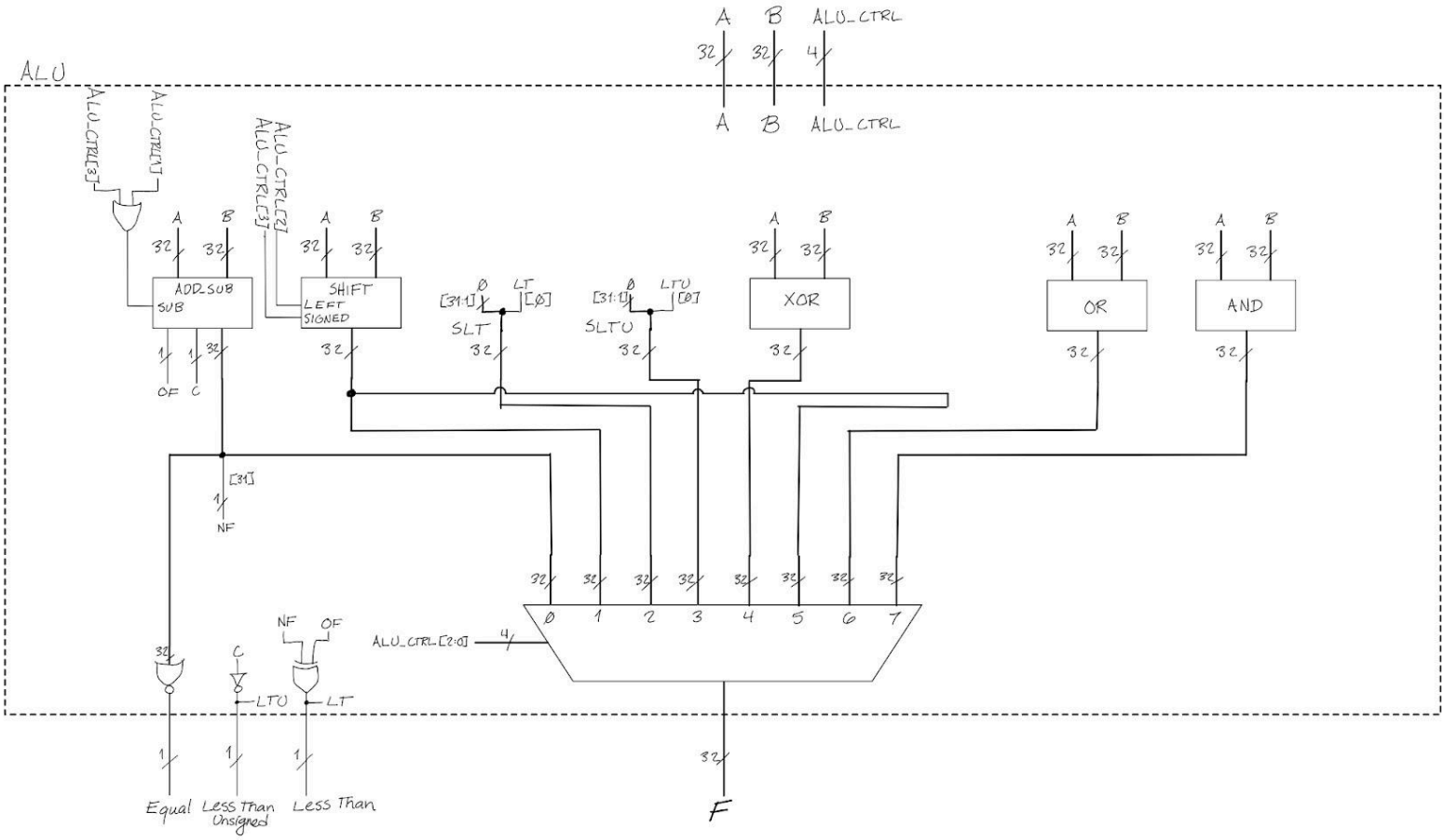
Additional Resources

I was not sure how to create a random number generating function for the ALU testbench using Verilog, so I asked Claude to create one for me. Since this is only a small part of the testbench and its addition greatly improves the thoroughness of the resulting ALU testbench, I find that this is a reasonable place to use generative AI.

[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your write-up.

Operations like add/subtract, and, or, xor, and the ALU's branching outputs like "less than", "less than unsigned", and "equal" are all thoroughly tested in the ALU's randomized unit test.

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?

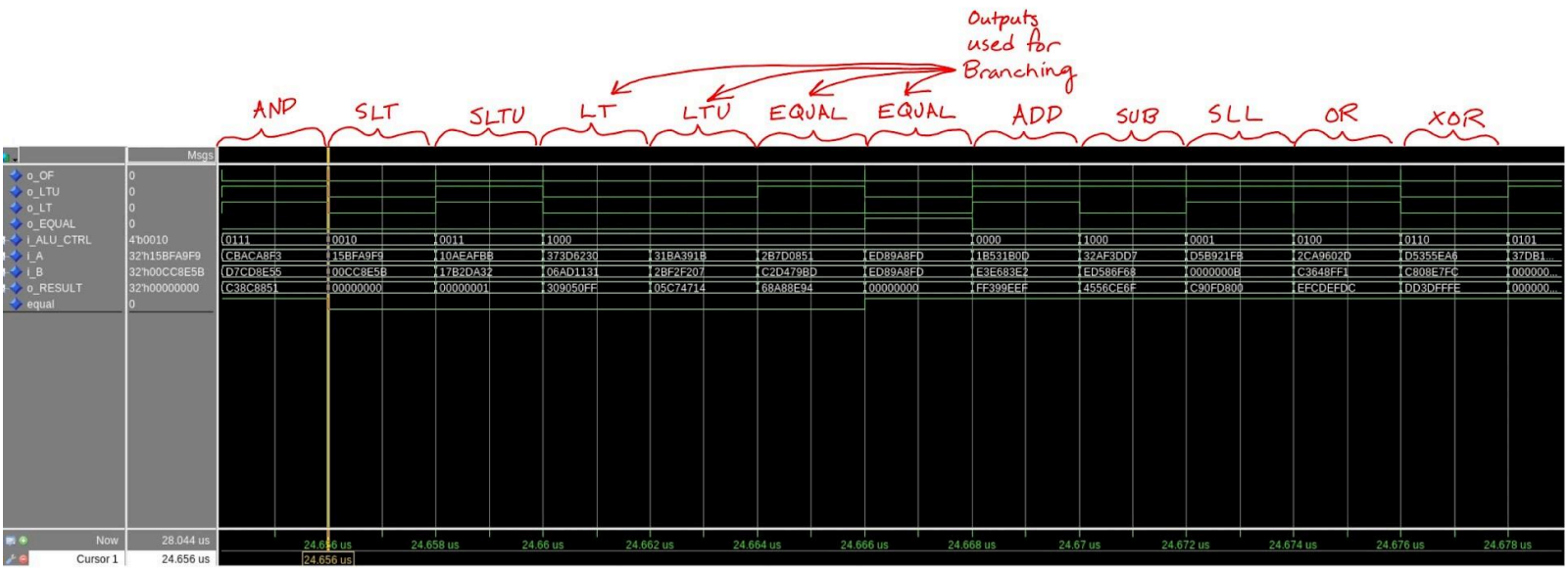


As indicated by the sketch above, the "zero" signal is calculated by NORing together all 32 bits from the output of the adder/subtractor. SLT is implemented by taking the XOR of the negative flag (NF) and the overflow flag (OF) signals and connecting this result to the first bit in the SLT function position of the function selector mux (shown at the

bottom of the ALU in the sketch above). Thus, whenever two numbers are subtracted, the LT flag is always automatically calculated.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your write-up.

An annotated waveform showing how the different ALU operations correspond to different parts of the waveform in the QuestaSim output:

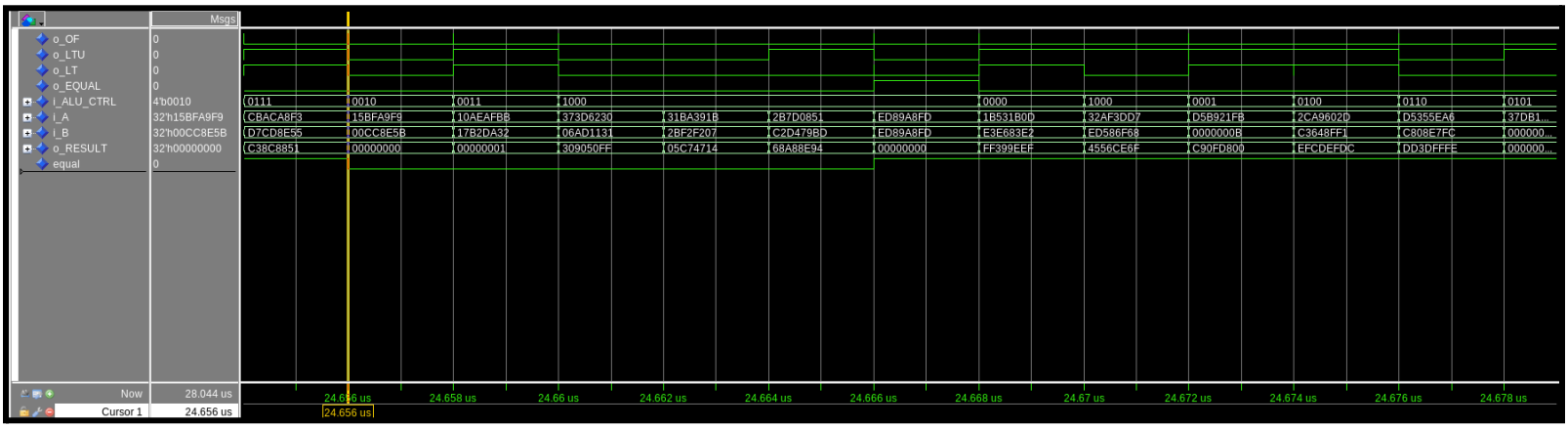


[Part 3.3.8] Justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

I added randomized testing to the ALU's testbench to verify this design's operation. I added a small function that generates a random 32-bit integer. The output of this random number generating function is then used as the input for the various test cases. I then ran the procedures 1000 times in a for loop and searched the console output using QuestaSim's output search to see whether any tests resulted in failures. The testbench used for the ALU is attached in Appendix B. A screenshot of console output resulting from running the ALU testbench is shown below.

```
# ** Note: AND_TEST PASSED: CBACA8F3 & D7CD8E55 = C38C8851 got: C38C8851
#    Time: 24656 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SLT_TEST PASSED: 15BFA9F9 < 00CC8E5B = 00000000 got: 00000000
#    Time: 24658 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -397597234 is not in bounds of subtype NATURAL.
#    Time: 24658 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -397597234 is not in bounds of subtype NATURAL.
#    Time: 24658 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -397597234 is not in bounds of subtype NATURAL.
#    Time: 24660 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SLTU_TEST PASSED: 10AEAFBB < 17B2DA32 = 00000001 got: 00000001
#    Time: 24660 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: LT_TEST PASSED: 373D6230 < 06AD1131 = 00000000 got: 00000000
#    Time: 24662 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -834287899 is not in bounds of subtype NATURAL.
#    Time: 24662 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -737341959 is not in bounds of subtype NATURAL.
#    Time: 24662 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -834287899 is not in bounds of subtype NATURAL.
#    Time: 24662 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -737341959 is not in bounds of subtype NATURAL.
#    Time: 24662 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -834287899 is not in bounds of subtype NATURAL.
#    Time: 24664 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -737341959 is not in bounds of subtype NATURAL.
#    Time: 24664 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: LTU_TEST PASSED: 31BA391B < 2BF2F207 = 00000000 got: 00000000
#    Time: 24664 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: EQ_TEST PASSED: 2B7D0851 = C2D479BD = 00000000 got: 00000000
#    Time: 24666 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: EQ_TEST PASSED: ED89A8FD = ED89A8FD = 00000001 got: 00000001
#    Time: 24668 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: ADD_TEST PASSED: 1B531B0D + E3E683E2 = FF399EEF got: FF399EEF
#    Time: 24670 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SUB_TEST PASSED: 32AF3DD7 - ED586F68 = 4556CE6F got: 4556CE6F
#    Time: 24672 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SLL_TEST PASSED: D5B921FB << 0000000B = C90FD800 got: C90FD800
#    Time: 24674 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: XOR_TEST PASSED: 2CA9602D ^ C3648FF1 = EFCDEFDC got: EFCDEFDC
#    Time: 24676 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: OR_TEST PASSED: D5355EA6 | C808E7FC = DD3DFFFE got: DD3DFFFE
#    Time: 24678 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SRL_TEST PASSED: 37DB124D >> 213EED3B = 00000006 got: 00000006
#    Time: 24680 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SRA_TEST PASSED: 3608511A >>> 00000003 = 06C10A23 got: 06C10A23
#    Time: 24682 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: AND_TEST PASSED: C81E8B80 & D34A85A9 = C00A8180 got: C00A8180
#    Time: 24684 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Note: SLT_TEST PASSED: 0BAA8800 < 2A05E202 = 00000001 got: 00000001
#    Time: 24686 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -181221849 is not in bounds of subtype NATURAL.
#    Time: 24686 ns  Iteration: 0  Instance: /alu_32bit_tb
# ** Warning: (vsim-86) Argument value -181221849 is not in bounds of subtype NATURAL.
```

The waveforms corresponding to these console outputs are shown in the screenshot below.

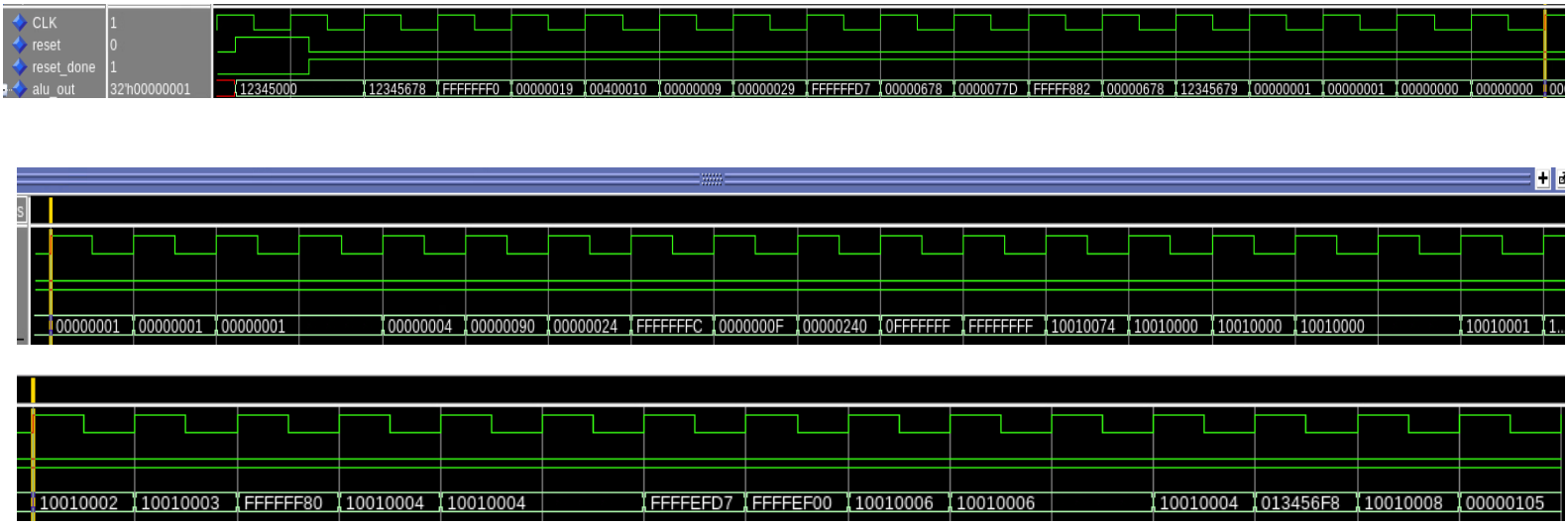


Randomized unit testing is the industry standard for professional digital designers. Since the inputs are random, I also tested the ALU's EQUAL output (used for branching) by explicitly calling the EQUAL_TEST procedure with equal but random input parameters.

4 Testing

[Part 4] In your write-up, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful also to annotate the waveforms directly.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.



Waveform of the base test that just runs through all the instructions and uses previous instructions to test future ones. The first waveform pic shows lui and some initialization and testing IMM's. Going through each cycle and ALU output:

1. lui t0, 0x12345 outputs 0x12345000
2. addi t0, t0, 0x678 reads the newly written t0 (0x12345000) and adds 0x678, producing 0x12345678
1. addi t1, x0, -16 sign-extends the 12-bit immediate -16 and outputs 0xFFFFFFF0
2. addi t2, x0, 25 outputs 0x00000019
3. auipc t3, 0 routes the current PC (0x00400010) through the ALU, outputting 0x00400010
4. add t4, t1, t2 computes -16 + 25 = 9 (0x00000009)
5. sub t5, t2, t1 computes 25 - (-16) = 41 (0x00000029)
6. sub t6, t1, t2 computes -16 - 25 = -41 (0xFFFFFFD7)
7. andi s0, t0, 0x7FF masks the lower 11 bits of 0x12345678, outputting 0x00000678
8. ori s1, s0, 0x105 computes 0x00000678 | 0x00000105 = 0x0000077D
9. xori s2, s1, -1 XORs 0x0000077D with sign-extended 0xFFFFFFFF to bitwise-invert s1, outputting 0xFFFFF882
10. and s3, t0, s1 computes 0x12345678 & 0x0000077D = 0x00000678
11. or s4, t0, t4 computes 0x12345678 | 0x00000009 = 0x12345679
12. xor s5, s4, t0 computes 0x12345679 ^ 0x12345678 = 0x00000001
13. slt s6, t1, t2 evaluates signed -16 < 25 as true (0x00000001)

14. slt s7, t2, t1 evaluates signed 25 < -16 as false (0x00000000)
15. sltu s8, t1, t2 treats t1 (0xFFFFFFF0) as unsigned and evaluates 0xFFFFFFF0 < 25 as false (0x00000000)

Now onto the next pic:

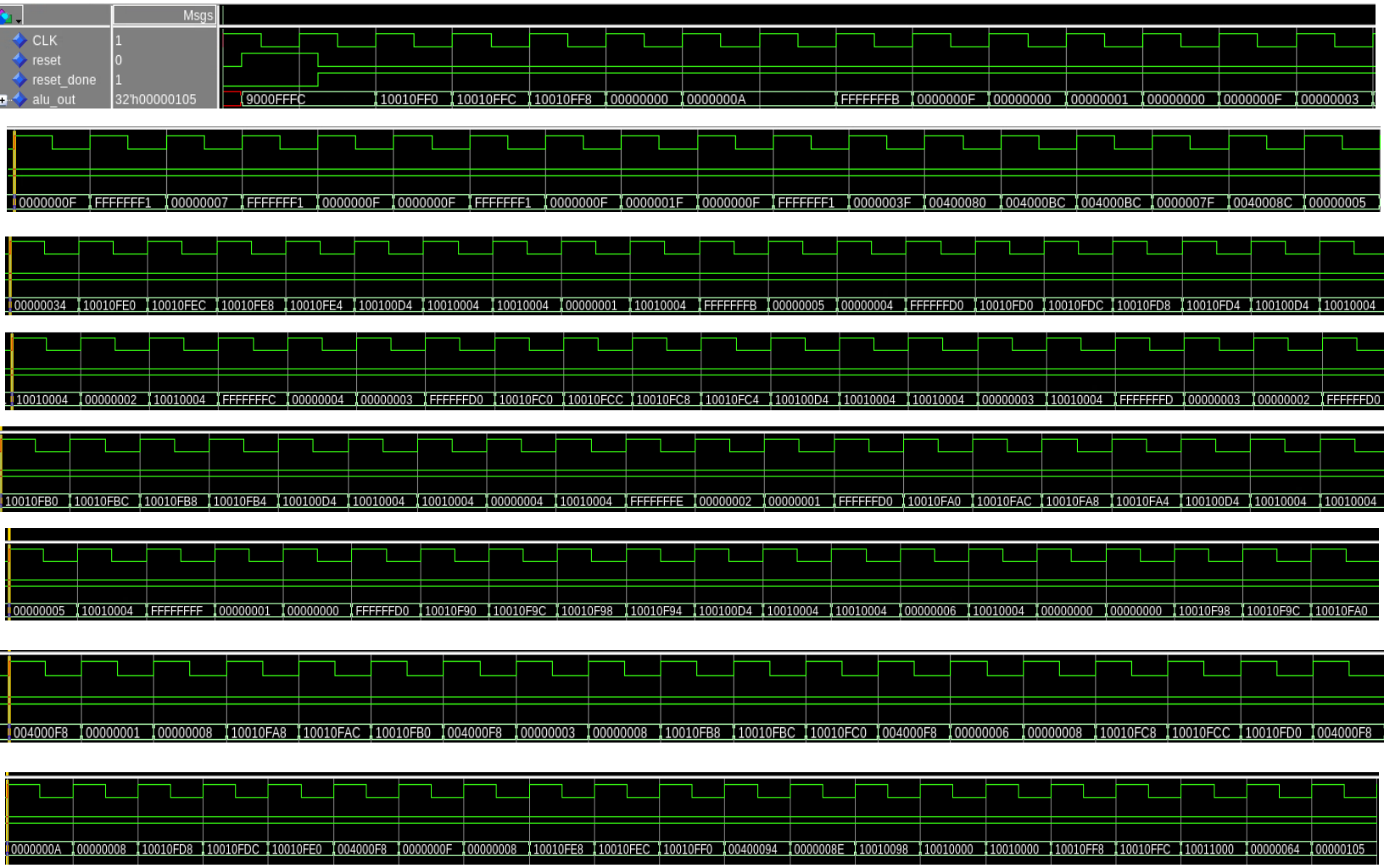
1. sltu s9, t2, t1 evaluates unsigned 25 < 0xFFFFFFF0 as true (0x00000001)
2. slti s10, t1, 0 evaluates signed -16 < 0 as true (0x00000001)
3. sltiu s11, t2, 100 and evaluate to true (0x00000001)
4. sltiu a0, t2, -1 evaluate to true (0x00000001)
5. addi a1, x0, 4 sets the register shift amount to 0x00000004
6. slli a2, t4, 4 shifts 9 left by 4 bits to produce 144 (0x00000090)
7. srli a3, a2, 2 logically shifts 144 right by 2 bits to produce 36 (0x00000024)
8. srai a4, t1, 2 arithmetically shifts -16 (0xFFFFFFF0) right by 2 bits with sign-extension, producing -4 (0xFFFFFFFC)
9. srli a5, t1, 28 logically shifts 0xFFFFFFF0 right by 28 bits, producing 15 (0x0000000F)
10. sll a6, a3, a1 shifts 36 left by a1 (4), producing 576 (0x00000240)
11. srl a7, t1, a1 logically shifts 0xFFFFFFF0 right by 4 bits, producing 0x0FFFFFFF
12. sra t4, t1, a1 arithmetically shifts 0xFFFFFFF0 right by 4 bits, producing -1 (0xFFFFFFFF)
13. la tp, mem_buf expands into auipc tp, 0xFC10, which outputs 0x10010074 and addi tp, tp, -116, which outputs 0x10010000, loading it into tp.
14. sw t0, 0(tp) and lw s0, 0(tp) both compute effective byte address 0x10010000 on alu_out
15. lb s1, 0(tp) and lbu s2, 1(tp) compute byte addresses 0x10010000 and 0x10010001 on alu_out

Waveform 3:

1. lb s3, 2(tp) and lbu s4, 3(tp) compute effective addresses 0x10010002 and 0x10010003 on alu_out
2. addi t5, x0, -128 generates 0xFFFFFF80 on alu_out
3. sb t5, 4(tp), lb s5, 4(tp), and lbu s6, 4(tp) compute address 0x10010004
4. lui t6, 0xFFFFF followed by addi t6, t6, -256 constructs 0xFFFFEF00 on alu_out (the write-back mux selects the upper immediate 0xFFFFF000 while alu out shows 0xFFFFEFD7)
5. sh t6, 6(tp), lh s7, 6(tp), and lhu s8, 6(tp) compute halfword address 0x10010006
6. lw s9, 4(tp) computes address 0x10010004
7. add s10, s0, s9 adds s0 (0x12345678) and s9 (0xEF000080) for 0x013456F8 on alu_out showing all the stores and loads worked.
8. sw s10, 8(tp) computes address 0x10010008

9. alu_out = 0x00000105 is the halt signal

[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.



Going through and annotating all of this would take forever, so I'm going to use 3 of the images. I'm going to be using the 2nd image after the initial state, the 6th image for the max recursion, and the final image for the final result.

Waveform 2 shows each of the 6 conditional branch instructions is tested first with a false condition (not taken) and then with a true condition (taken) while setting a unique bit in accumulator s0 (+1, +2, +4, +8, +16, +32). In the waveform, the ALU computes rs1 - rs2 (0x0000000F for 10 - (-5) and 0xFFFFFFF1 for -5 - 10) to drive o_EQUAL, o_LT, and o_LTU, culminating in alu_out = 0x0000003F (63), which confirms all 6 branch types passed without triggering br_fail. Following this la t3, helper_fn (0x004000BC) and jalr ra, t3, 0 jump to helper_fn, which adds 64 to produce alu_out = 0x0000007F (127)

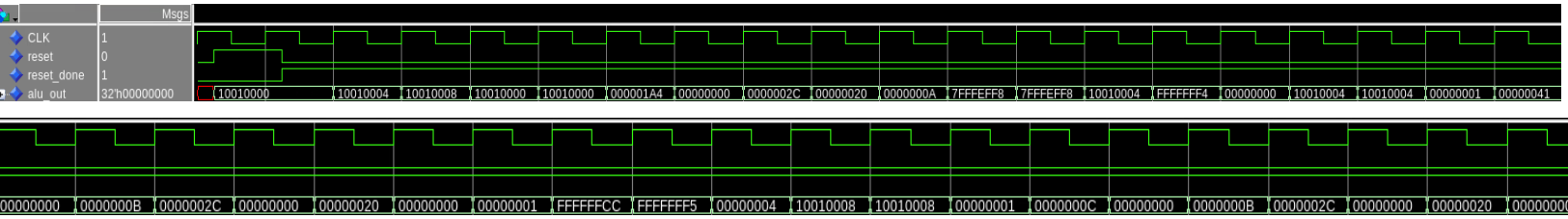
and returns via jalr x0, ra, 0 (0x0040008C) before loading a0 = 5 (0x00000005) for recurse_sum(5).

After this in the next waveforms sp decrements by 16 bytes (0x10) per call down to 0x10010F90 and unwinds back to 0x10011000 so 4 cycles show the sp 10010FE0 ,10010FD0 , 10010FC0 , 10010FB0 , 10010FA0 , 10010F90 each of these go from 0 to C… end at 4. Alongside this each recursion also shows its depth like 00000003. Waveform 6 shows this after Depth 5 increments depth_reached to 00000005, jal ra, recurse_sum (FFFFFFD0) enters Depth 6 (n = 0), allocating the 6th recursive activation record at sp = 10010F90, saving ra, s1, and a0 (10010F9C, 10010F98, 10010F94), and writing 00000006 to depth_reached (10010004, 0000006) The base-case branch

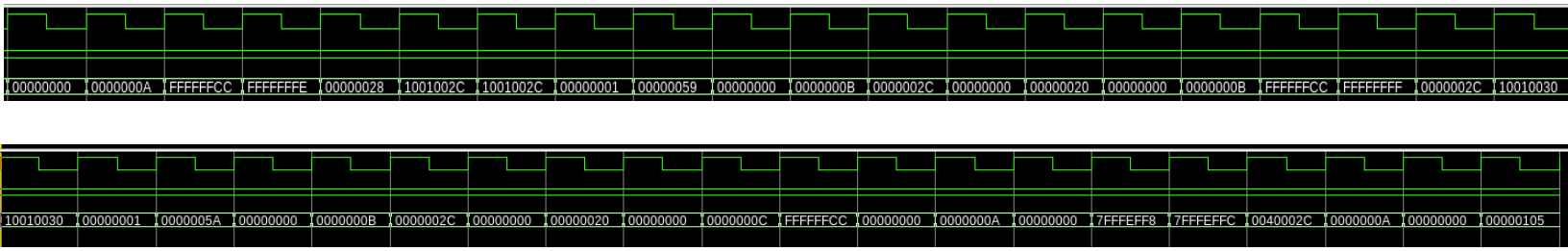
bge x0, a0, recurse_base evaluates 0 - 0 = 00000000 (taken), sets a0 = 00000000, and pops the Depth 6 frame (10010F98, 10010F9C, sp = 10010FA0).

In waveform 8 the last one, Depth 2 computes 4 + 6 = 0000000A and restores sp = 10010FE0, and Depth 1 computes 5 + 10 = 0000000F (15) and restores sp = 10010FF0 before returning to main via jalr x0, ra, 0 (00400094). Finally, main adds the branch/jump status (127) and recursive sum (15) to produce 0000008E (142), stores it to cf_result (10010000), pops main's stack frame back to 10011000, jumps to halt (00000064), and halts on wfi (00000105).

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.



……skip like 35000 ns



This shows the mergesort waveform mainly showing the initial cycles and then the final cycles as there is about 36000ns and that would take forever to capture and be unreadable.

[65, 12, 10, 89, 11, 70, 67, 5, 9, 45, 90, 7] in ascending order ([5, 7, 9, 10, 11, 12, 45, 65, 67, 70, 89, 90]) using recursive Merge Sort (sort, msort, and merge) and stores the sorted result back into the original array memory buffer (0x10010004–0x10010030).

main loads the base address of array via la a0, array (10010000 ,10010004) and loads array_size = 12 from 0x10010000 via la t0, array_size (10010008 ,10010000) and lw a1, 0(t0) (10010000). Calling jal ra, print_array (000001A4) initializes the loop counter t0 = 0 (00000000) and ASCII delimiters ',' (0000002C), ' ' (00000020), and '\n' (0000000A), pushes a0 onto the stack at sp = 7FFFEFF8, and copies the base pointer into t4 = 10010004. After that, this continues into the second waveform image: each iteration checks bge t0, a1, print_array_end (FFFFFFF4 for i = 0, FFFFFFF5 for i = 1), shifts the index by 2 (slli t5, t0, 2), adds the base address 10010004, and loads the element from memory. Waveform 1 shows lw t5, 0(t5) at 10010004 loading array[0] = 65 (00000041), and Waveform 2 shows the next iteration (i = 00000001, jump FFFFFFCC) loading array[1] = 12 (0000000C) from 10010008, with each element followed by the 6-cycle ecall sequence (00000000, 0000000B, 0000002C, 00000000, 00000020, 00000000) to print "65, 12, ..." before main calls jal ra, sort.

In the final two waveforms, it shows the verified sort and end. Once sort finishes recursively splitting (msort) and merging via temp_array back into array (copy_back_loop), main invokes print_array a second time. In Waveform 3, loop index i = 10 (0000000A) passes bge (10 - 12 =

FFFFFFFE), computes offset 10 << 2 = 00000028 and address 1001002C, and loads array[10] = 89 (00000059) confirming that 89 (originally at index 3) was moved to index 10. The loop then increments i to 11 (0000000B), passes bge (11 - 12 = FFFFFFFF), computes offset 0000002C and address 10010030, and in Waveform 4 loads the maximum value array[11] = 90 (0000005A) (originally at index 10) into a0 for printing. After printing 90, t0 increments to 12 (0000000C) and jumps (FFFFFFCC) to bge t0, a1, print_array_end, which evaluates 12 - 12 = 00000000 (o_LT = 0, branch taken). The processor prints the final newline (0000000A, 00000000), restores a0 from the stack (7FFFEFF8) and deallocates the frame (sp = 7FFFEFFC), returns to main via ret (0040002C), executes li a7, 10 (0000000A) and ecall (00000000), and cleanly halts on wfi (00000105), passing the RARS trace comparison.

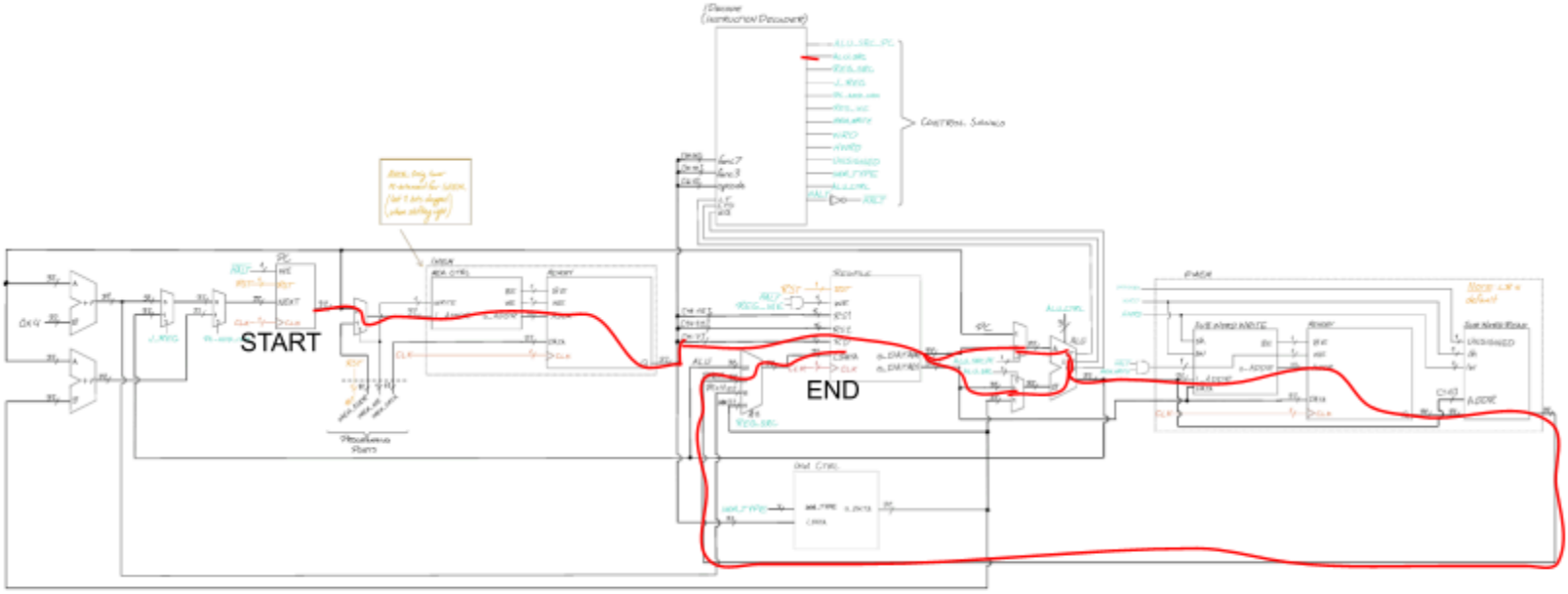
5 Synthesis

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

FMax: 23.28mhz Clk Constraint: 20.00ns Slack: -22.96ns

Critical path starts at the PC register and ends at the reg file as data input.

It flows from the PC to the Imem load mux, then to the Imem, then to the Regfile/IDecode, but in the log it all got “merged” and showed up as 6 ALU_SRC_PC_MUXes, which is the next part, and then it goes through the ALU to the DMem and the subword read, then to the write-back reg src mux and regfile write as input.



4 things that could speed up the processor by allowing a higher max frequency

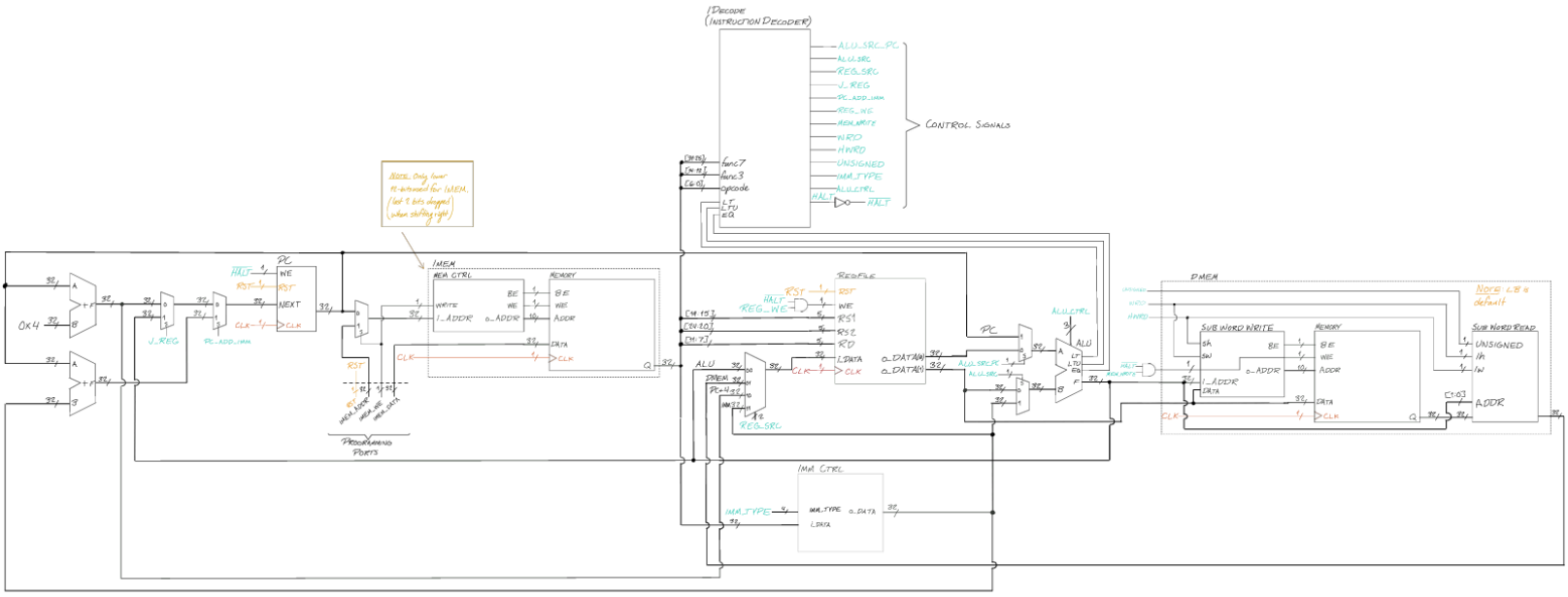
- Pipelining the Single-Cycle Datapath 3 or 4x speed up.
    - This is not really a component but would provide a massive speed up so I had to include it plus we have to add it in part 2 anyways.
- IMem and DMem — 57.2% of total delay.(12.997ns for IMEM and 11.554 ns for DMEM)
    - In the log the Memory components take up more than half the entire 42.962ns delay and I don’t know if it's something we can mess with but

focusing on improving the DMem component would give us the most gains in reducing the delay and improving the Fmax.

- ALU Shifter DMem Address — 14.4% of delay.
    - This is something that showed up in the log but even though memory calculation in the ALU is just a simple add the STA is evaluating the longest path and thus including the barrel shifter portion and the mux since its output is a single bus there might be a way to restructure so it doesn't have this delay show up.
- Register File Read Mux & ALU_SRC_PC_MUX Decode (13.0% of delay).
    - 5.588ns is taken between the Imem output and the ALU src mux input so the amount of time for that regfile read and decode to complete. Something weird in the output log for the synthesis is that it goes from IMEM then like 6 cascaded mux's
      16.780 0.281 FF CELL IMem|Mux13~681|combout 18.491 1.711 FF IC ALU_SRC_PC_MUX|o_DATA[26]~116|datab 18.916 0.425 FF CELL ALU_SRC_PC_MUX|o_DATA[26]~116|combout 19.186 0.270 FF IC ALU_SRC_PC_MUX|o_DATA[26]~117|datab 19.611 0.425 FF CELL ALU_SRC_PC_MUX|o_DATA[26]~117|combout
    - So clearly its doing some kind of merging/optimization that goes beyond component boundaries and turns a big cascade to get the ALU input src mux result. Regfile and Decode don't show up. Somehow merging or removing the ALU_SRC_PC_MUX should fix this and speed up this section and this should be possible as its only used for auipc and we could move elsewhere or merge it with one of the branch jump pc adder's.

Appendix A

Single-cycle processor schematic:



Appendix B

alu_32bit_tb.v

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.NUMERIC_STD.all;
use ieee.math_real.all;


use work.std_logic_vector_array.all;

library  std;
use std.env.all;

entity ALU_32BIT_tb is
    generic(
        eighth_clk      : time      := 1.25 ns;
        quarter_clk     : time      := eighth_clk * 2;
        half_clk        : time      :=  quarter_clk * 2;
        full_clk        : time      :=  half_clk * 2;
        BITS            : integer   := 32
    );
end ALU_32BIT_tb;

architecture rtl of ALU_32BIT_tb is

    constant s_ALU_ADD    : STD_LOGIC_VECTOR(3 downto 0) := "0000";
    constant s_ALU_SUB    : STD_LOGIC_VECTOR(3 downto 0) := "1000";
    constant s_ALU_SLL    : STD_LOGIC_VECTOR(3 downto 0) := "0001";
    constant s_ALU_SLT    : STD_LOGIC_VECTOR(3 downto 0) := "0010";
    constant s_ALU_SLTU   : STD_LOGIC_VECTOR(3 downto 0) := "0011";
    constant s_ALU_XOR    : STD_LOGIC_VECTOR(3 downto 0) := "0100";
    constant s_ALU_SRL    : STD_LOGIC_VECTOR(3 downto 0) := "0101";
    constant s_ALU_SRA    : STD_LOGIC_VECTOR(3 downto 0) := "1101";
    constant s_ALU_OR     : STD_LOGIC_VECTOR(3 downto 0) := "0110";
    constant s_ALU_AND    : STD_LOGIC_VECTOR(3 downto 0) := "0111";

    signal i_ALU_CTRL       : STD_LOGIC_VECTOR(3 downto 0);
    signal i_A              : STD_LOGIC_VECTOR(BITS-1 downto 0);
    signal i_B              : STD_LOGIC_VECTOR(BITS-1 downto 0);
    signal o_RESULT         : STD_LOGIC_VECTOR(BITS-1 downto 0);
    signal LESS_UNSIGNED    : STD_LOGIC;
```

```
    signal LESS             : STD_LOGIC;
    signal equal            : STD_LOGIC;
    signal o_EQUAL          : STD_LOGIC := '0';
    signal o_LT             : STD_LOGIC := '0';
    signal o_LTU            : STD_LOGIC := '0';
    signal o_OF             : STD_LOGIC := '0';
    signal s_A              : STD_LOGIC_VECTOR(BITS-1 downto 0);
    signal s_B              : STD_LOGIC_VECTOR(BITS-1 downto 0);

    component ALU_32BIT is
        generic (
            DATA_WIDTH : INTEGER := 32
        );
        port (
            i_ALU_CTRL  : in STD_LOGIC_VECTOR(3 downto 0);
            -- ALU_CTRL BIT MEANINGS:
            -- ALU_CTRL[3]      : ('1' : SHIFT_RIGHT_SIGNED or SUBTRACT,
'0' : UNSIGNED or ADD)
            -- ALU_CTRL[2:0]    :  (000 -> ADD/SUB
            --                      001 -> SLL
            --                      010 -> SLT  (set less than)
            --                      011 -> SLTU (set less than unsigned)
            --                      100 -> XOR
            --                      101 -> SRL or SRA
            --                      110 -> OR
            --                      111 -> AND)
            i_A         : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
            i_B         : in STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
            o_RESULT    : out STD_LOGIC_VECTOR(DATA_WIDTH - 1 downto 0);
            o_EQUAL     : out STD_LOGIC;
            o_LT        : out STD_LOGIC;
            o_LTU       : out STD_LOGIC;
            o_OF        : out STD_LOGIC
        );
    end component;

begin

    ALU_inst: ALU_32BIT
    generic map(
        DATA_WIDTH => BITS
    )
```

```
port map(
    i_ALU_CTRL => i_ALU_CTRL,
    i_A => i_A,
    i_B => i_B,
    o_RESULT => o_RESULT,
    o_EQUAL => o_EQUAL,
    o_LT => o_LT,
    o_LTU => o_LTU,
    o_OF => o_OF
);

EQUAL           <= '1' when unsigned(s_A) = unsigned(s_B) else '0';
LESS            <= '1' when signed(s_A) < signed(s_B)  else '0';
LESS_UNSIGNED   <= '1' when unsigned(s_A) < unsigned(s_B)  else '0';

p_test_cases: process
    variable EQUAL_VAR   : INTEGER := 0;

    procedure CHECK(
        constant TEST       : in STRING;
        constant OPERAND    : in STRING;
        constant A          : in STD_LOGIC_VECTOR(BITS-1 downto 0);
        constant B          : in STD_LOGIC_VECTOR(BITS-1 downto 0);
        EXPECTED            : in STD_LOGIC_VECTOR(BITS-1 downto 0);
        RESULT              : in STD_LOGIC_VECTOR(BITS-1 downto 0)
    ) is
    begin

        if (EXPECTED /= RESULT) then
            report  TEST & " FAILED: " &
                    to_hstring(A) &
                    " " & OPERAND & " " &
                    to_hstring(B) &
                    " = " &
                    to_hstring(EXPECTED) &
                    " but got: " &
                    to_hstring(RESULT)
            severity FAILURE;
        else
            report  TEST & " PASSED: " &
                    to_hstring(A) &
                    " " & OPERAND & " " &
```

```
                        to_hstring(B) &
                        " = " &
                        to_hstring(EXPECTED) &
                        " got: " &
                        to_hstring(RESULT)
                severity NOTE;
            end if;

        end procedure;

        variable seed1 : positive := 1;
        variable seed2 : positive := 42;

        impure function RANDOM_INT return integer is
            variable rand_real : real;
            constant rand_min : integer := -2**30;
            constant rand_max : integer :=  2**30 - 1;   -- keeps A+B and
A-B inside integer range
        begin
            uniform(seed1, seed2, rand_real);

            -- 2. Scale it to the desired integer range
            return integer(trunc(rand_real * (real(rand_max) -
real(rand_min) + 1.0))) + rand_min;
        end function;

        procedure ADD_TEST(
            constant A     : in INTEGER;
            constant B     : in INTEGER
        ) is
        begin
            i_ALU_CTRL  <= s_ALU_ADD;
            i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
            i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

            wait for quarter_clk;
            CHECK(
                "ADD_TEST",
                "+",
                STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
                STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
                STD_LOGIC_VECTOR(TO_SIGNED(A + B, 32)),
```

```
            o_RESULT
        );

    end procedure;

    procedure SUB_TEST(
        constant A     : in INTEGER;
        constant B     : in INTEGER
        ) is
    begin
        i_ALU_CTRL  <= s_ALU_SUB;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "SUB_TEST",
            "-",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A - B, 32)),
            o_RESULT
        );
    end procedure;

    procedure SLL_TEST(
        constant A      : in INTEGER;
        constant SHIFT  : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_SLL;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(abs(SHIFT) mod 32,
32));

        wait for quarter_clk;
        CHECK(
            "SLL_TEST",
            "<<",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(abs(SHIFT) mod 32, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)) sll abs(SHIFT) mod 32,
```

```
            o_RESULT
        );
    end procedure;

    procedure XOR_TEST(
        constant A     : in INTEGER;
        constant B     : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_XOR;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "XOR_TEST",
            "^",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)) XOR
STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            o_RESULT
        );
    end procedure;

    procedure OR_TEST(
        constant A     : in INTEGER;
        constant B     : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_OR;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "OR_TEST",
            "|",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)) OR
STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
```

```
            o_RESULT
        );
    end procedure;

    procedure SRL_TEST(
        constant A      : in INTEGER;
        constant SHIFT  : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_SRL;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(abs(SHIFT) mod 32,
32));

        wait for quarter_clk;
        CHECK(
            "SRL_TEST",
            ">>",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(SHIFT, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32) srl abs(SHIFT) mod 32),
            o_RESULT
        );
    end procedure;

    procedure SRA_TEST(
        constant A      : in INTEGER;
        constant SHIFT  : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_SRA;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(abs(SHIFT) mod 32,
32));

        wait for quarter_clk;
        CHECK(
            "SRA_TEST",
            ">>>",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(abs(SHIFT) mod 32, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32) sra abs(SHIFT) mod 32),
```

```
            o_RESULT
        );
    end procedure;

    procedure AND_TEST(
        constant A     : in INTEGER;
        constant B     : in INTEGER
    ) is
    begin
        i_ALU_CTRL  <= s_ALU_AND;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "AND_TEST",
            "&",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)) AND
STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            o_RESULT
        );

    end procedure;

    procedure SLT_TEST(
        constant A     : in INTEGER;
        constant B     : in INTEGER
    ) is
    begin
        s_A <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        s_B <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        i_ALU_CTRL  <= s_ALU_SLT;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "SLT_TEST",
            "<",
```

```
        STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
        STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
        (0 => LESS, others => '0'),
        o_RESULT
    );

end procedure;

procedure SLTU_TEST(
    constant A     : in INTEGER;
    constant B     : in INTEGER
) is
begin
    s_A <= STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32));
    s_B <= STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32));

    i_ALU_CTRL  <= s_ALU_SLTU;
    i_A         <= STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32));
    i_B         <= STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32));

    wait for quarter_clk;
    CHECK(
        "SLTU_TEST",
        "<",
        STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32)),
        STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32)),
        (0 => LESS_UNSIGNED, others => '0'),
        o_RESULT
    );

end procedure;

procedure LT_TEST(
    constant A     : in INTEGER;
    constant B     : in INTEGER
) is
begin
    s_A <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
    s_B <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

    i_ALU_CTRL  <= s_ALU_SUB;
    i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
```

```
    i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

    wait for quarter_clk;
    CHECK(
        "LT_TEST",
        "<",
        STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
        STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
        (0 => LESS, others => '0'),
        (0 => o_LT, others => '0')
    );

end procedure;

procedure LTU_TEST(
    constant A     : in INTEGER;
    constant B     : in INTEGER
) is
begin
    s_A <= STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32));
    s_B <= STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32));

    i_ALU_CTRL  <= s_ALU_SUB;
    i_A         <= STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32));
    i_B         <= STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32));

    wait for quarter_clk;
    CHECK(
        "LTU_TEST",
        "<",
        STD_LOGIC_VECTOR(TO_UNSIGNED(A, 32)),
        STD_LOGIC_VECTOR(TO_UNSIGNED(B, 32)),
        (0 => LESS_UNSIGNED, others => '0'),
        (0 => o_LTU, others => '0')
    );

end procedure;

procedure EQ_TEST(
    constant A     : in INTEGER;
    constant B     : in INTEGER
) is
```

```
    begin
        s_A <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        s_B <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        i_ALU_CTRL  <= s_ALU_SUB;
        i_A         <= STD_LOGIC_VECTOR(TO_SIGNED(A, 32));
        i_B         <= STD_LOGIC_VECTOR(TO_SIGNED(B, 32));

        wait for quarter_clk;
        CHECK(
            "EQ_TEST",
            "=",
            STD_LOGIC_VECTOR(TO_SIGNED(A, 32)),
            STD_LOGIC_VECTOR(TO_SIGNED(B, 32)),
            (0 => EQUAL, others => '0'),
            (0 => o_EQUAL, others => '0')
        );

    end procedure;

begin

    -- Tests
    for i in 0 to 1000 loop
        ADD_TEST(   RANDOM_INT, RANDOM_INT);
        SUB_TEST(   RANDOM_INT, RANDOM_INT);
        SLL_TEST(   RANDOM_INT, RANDOM_INT);
        XOR_TEST(   RANDOM_INT, RANDOM_INT);
        OR_TEST(    RANDOM_INT, RANDOM_INT);
        SRL_TEST(   RANDOM_INT, RANDOM_INT);
        SRA_TEST(   RANDOM_INT, RANDOM_INT);
        AND_TEST(   RANDOM_INT, RANDOM_INT);
        SLT_TEST(   RANDOM_INT, RANDOM_INT);
        SLTU_TEST(  RANDOM_INT, RANDOM_INT);
        LT_TEST(    RANDOM_INT, RANDOM_INT);
        LTU_TEST(   RANDOM_INT, RANDOM_INT);
        EQ_TEST(    RANDOM_INT, RANDOM_INT);
        EQUAL_VAR := RANDOM_INT;
        EQ_TEST(    EQUAL_VAR, EQUAL_VAR);
    end loop;
    wait for full_clk;
    wait for full_clk;
```

```
        stop;

    end process;

end rtl;
```